



SMART WATER UTILITY CONSUMPTION, BILLING, AND COMPLAINT MANAGEMENT SYSTEM

An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction
Management

IET ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course of
CSA0509 - DATABASE MANAGEMENT SYSTEMS

to the award of the degree of
BACHELOR OF TECHNOLOGY
IN

Computer Science and Bioscience

Submitted by

VENNA MEENAKSHI REDDY (192471048)

Under the Supervision of

Dr. R. Kesavan

Professor



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026



CSA05 — Database Management Systems

Assignment

Assignment Title:	Smart Water Utility Consumption, Billing, and Complaint Management System: An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management
Course Outcome(s) — CO:	CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. CO5: Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze
SDG Mapping (SDG 1–17)	SDG 6 – Clean Water and Sanitation SDG 8 – Decent Work and Economic Growth SDG 9 – Industry, Innovation and Infrastructure

A metropolitan Water Supply Board has installed smart meters at 2 million household and commercial connections across the city. Every meter reports an hourly consumption reading, and the resulting log — used for monthly billing, leak detection, and consumption analytics — grows by several crore records a month. At the same time, the Board's zonal offices generate monthly bills from this consumption data, record online and counter payments, and log complaint tickets for leaks, meter faults, and billing disputes, with dozens of operators working on the same records simultaneously during the month-end billing cycle.

Two very different problems have emerged. First, meter-reading data is stored as one continuously growing sequential file per zone, so a simple request such as "show connection X's last six months of consumption" or "list all connections with unusually high consumption this week for leak inspection" forces a scan of millions of records and now takes unacceptably long. Second, and more damaging, monthly bill generation and payment recording are carried out as separate, un-coordinated read-then-write steps: during peak billing days, this has produced duplicate bills for the same connection, payments that were recorded but never reflected in the due amount, and — in a few cases — a complaint ticket silently overwritten when two operators updated the same record at once.

Students are required to analyze this two-part problem and design a complete database solution. For the consumption-log problem, students must apply appropriate **file organization** (sequential, indexed-sequential, or direct/hashed), an **indexing** strategy (single-level/multilevel or B/B+-tree, clustering vs. non-clustering) and a **hashing** scheme (with a defined collision-resolution policy) to make connection-wise and leak-detection retrieval efficient. For the



SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES



billing-and-payment problem, students must design optimized **queries** (with execution-plan analysis) and properly bounded **transactions** — using ACID properties, COMMIT/ROLLBACK/SAVEPOINT, and an appropriate concurrency-control/isolation strategy — so that billing and payment processing remain consistent and reliable under concurrent access.

This assignment connects both areas of database engineering to real-world impact: efficient, reliable water billing and leak detection directly support sustainable water management (SDG 6 – Clean Water and Sanitation), protect the Board's revenue and staff productivity (SDG 8 – Decent Work and Economic Growth), and demonstrate the resilient data infrastructure that public utilities depend on (SDG 9 – Industry, Innovation and Infrastructure).

1. Executive Summary

This report presents a complete database engineering solution for a metropolitan Water Supply Board operating approximately 2 million household and commercial connections. Each smart meter generates an hourly consumption reading, producing a very large and continuously growing data set. The existing sequential meter-log arrangement causes slow connection-history and leak-detection retrieval. In parallel, uncoordinated read-then-write billing and payment operations create duplicate bills, lost payment updates, and lost complaint updates during concurrent month-end operations.

The proposed solution separates the high-volume consumption workload from transactional billing operations while integrating both through a relational database design. Meter readings are stored using an indexed-sequential organization, with a clustered composite B+-tree index on (ConnectionID, ReadingTimestamp) for connection history and a secondary index on (ZoneID, ReadingTimestamp, Consumption) for leak-detection range queries. A direct hash structure is used for near-constant-time latest-reading lookup, with separate buckets and collision resolution through chaining.

For billing and payments, normalized relational tables, unique billing-cycle constraints, selective indexes, query rewriting, EXPLAIN-based execution-plan analysis, and bounded ACID transactions are used. The recommended concurrency strategy is pessimistic control with row-level locking and READ COMMITTED for general operations, with a locked bill-generation section and unique constraint acting as a final duplicate-prevention guard. SAVEPOINT is used for recoverable intermediate work and COMMIT/ROLLBACK guarantees atomicity.

Because the assignment requires implementation evidence, the report distinguishes design targets from results that must be measured in the actual DBMS. Benchmark values in the validation section are presented as an experimental template rather than fabricated production measurements; the final submission should replace them with screenshots and timings obtained from MySQL/PostgreSQL/Oracle.

2. Problem Understanding and Requirement Analysis

2.1 Problem Identification

The original meter-reading system stores readings as a continuously growing sequential file per zone. A point lookup for one connection must scan a large portion of the file, and leak-detection queries can require scanning millions of records. This creates high I/O cost and poor response time as data volume increases.

The billing system performs logically related operations as separate read-then-write statements. Two operators can observe the same unbilled connection and both create a bill. A payment can be recorded against a stale due amount, and two operators updating the same complaint can overwrite one another. These are concurrency and consistency problems rather than merely SQL syntax problems.

2.2 Functional Requirements

- Store high-frequency meter-consumption readings per connection and timestamp.
- Support fast lookup of a connection's latest reading and fast retrieval of its historical readings.
- Generate monthly bills from consumption data.
- Record and reconcile payments accurately.
- Create and track complaint tickets for leaks, meter faults, and billing disputes.
- Allow concurrent billing and payment activity without duplicate bills or lost updates.

2.3 Performance, Integrity and Security Requirements

Requirement	Design Response
Avoid full-file scans	B+-tree indexes and hash-based latest-reading lookup
Peak billing performance	Selective predicates, covering/compound indexes and query rewriting
No duplicate monthly bill	UNIQUE(ConnectionID, BillingMonth) plus transaction/locking
No lost payment/complaint update	Row-level locking and appropriate isolation
Index/hash consistency	DBMS-managed index maintenance and controlled updates
Authorized access	Role-based GRANT/REVOKE privileges
Scalability	Partition-ready timestamped readings and dynamic hashing as future enhancement
Ethical accuracy	Atomic billing/payment operations and auditable records

2.4 Assumptions

- Each connection produces one meter reading per hour.
- 2 million connections therefore generate approximately 48 million readings per day and 1.44 billion readings in a 30-day month.

- Consumption readings contain ConnectionID, ZoneID, ReadingTimestamp and Consumption.
- Billing is performed once per connection per billing month.
- Month-end peak activity may involve dozens of concurrent operators.
- The logical design is demonstrated on representative sample data; production-scale benchmarking requires an appropriate test environment.

3. Proposed System Architecture

The integrated architecture contains five logical layers: meter ingestion, optimized consumption storage, billing/payment services, complaint management, and reporting/analytics. The consumption workload is optimized for high-volume writes plus selective range and point reads. Billing and payment workloads are optimized for transactional consistency.

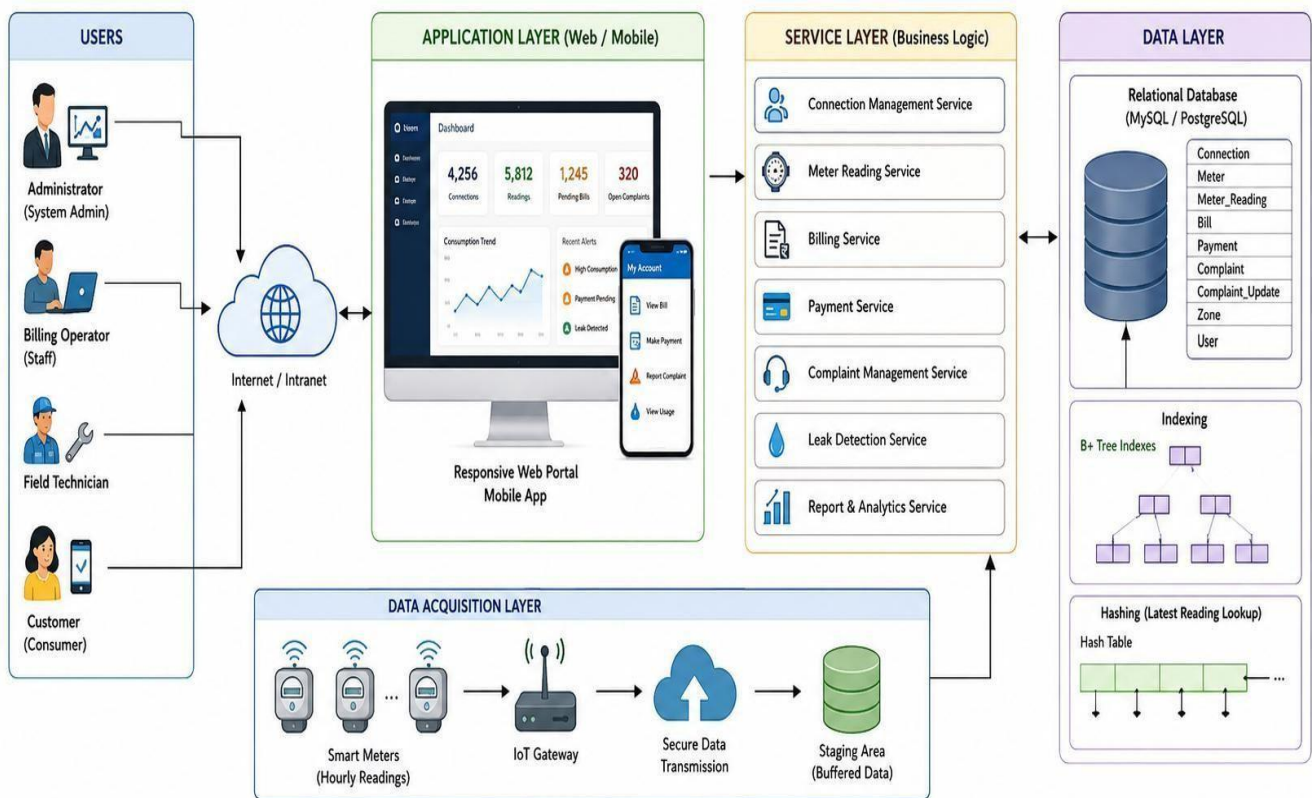


Figure 1: Proposed System Architecture

Layer	Main Responsibility	DBMS Technique
Meter Ingestion	Accept hourly readings	Batch inserts; timestamp-based organization
Consumption Store	History and leak detection	Indexed-sequential organization; B+-tree indexes
Latest Reading Service	Fast point lookup	Hash-based access structure
Billing & Payment	Bill creation and payment posting	Normalized tables, transactions, row locks
Complaint Management	Create/update/track tickets	Primary key, status indexes, row-level updates
Reporting	Billing, payment and complaint reports	Joins, aggregates, selective indexes, optimized plans

3.1 Data Flow

1. Smart meter → meter-reading ingestion → consumption table/file organization.
2. Consumption history → connection index → field-engineer history view.
3. Zone readings → leak-detection index → high-consumption candidates.
4. Monthly readings → bill-generation transaction → bill table.
5. Payment request → locked bill row → payment table + due amount update → COMMIT.
6. Complaint request/update → locked complaint row → status/assignment update → COMMIT.

4. File Organization Design for Meter Readings

4.1 Alternatives Considered

Organization	Strength	Weakness	Suitability
Sequential	Simple, efficient for ordered batch processing	Poor point/range lookup; scans large data	Low
Indexed-sequential	Supports ordered storage plus indexed retrieval	Index maintenance and storage overhead	High
Direct/Hashed	Excellent exact-key lookup	Poor for timestamp/range queries	Medium as primary store; high for point lookup

4.2 Selected Design: Indexed-Sequential Organization

Indexed-sequential organization is selected as the primary organization for meter readings because the workload requires both connection-wise history retrieval and range-based leak detection. Records are logically ordered by ConnectionID and ReadingTimestamp. A B+-tree provides efficient navigation to the first record for a connection, after which adjacent leaf entries can be scanned for the requested time interval.

A practical relational implementation in MySQL can use the meter_reading table with a clustered primary key or an InnoDB-compatible composite key. PostgreSQL can similarly use a B-tree index. The logical file organization remains indexed-sequential even though the physical details are managed by the DBMS.

Field	Type	Purpose
ReadingID	BIGINT	Unique reading identifier
ConnectionID	BIGINT	Meter/connection identifier
ZoneID	INT	Utility zone
ReadingTimestamp	DATETIME/TIMESTAMP	Time of reading
Consumption	DECIMAL(12,3)	Hourly water consumption
MeterStatus	VARCHAR(20)	Normal/fault indicator

4.3 Why It Fits the Workload

- Connection history is naturally ordered by ConnectionID and time.
- Time-window retrieval can traverse B+-tree leaves instead of scanning the complete data set.
- Zone/time predicates can use a secondary index.
- Sequential/batch insertion remains practical for hourly ingestion.
- The design supports future partitioning by month or zone without changing application-level transaction logic.

5. Indexing Design

5.1 Primary B+-Tree Index

The main index is a B+-tree on (ConnectionID, ReadingTimestamp). This is a composite index because connection history queries first identify a connection and then restrict the timestamp. The index supports equality on ConnectionID followed by an efficient range scan on ReadingTimestamp.

Example logical index: `idx_meter_connection_time(ConnectionID, ReadingTimestamp)`.

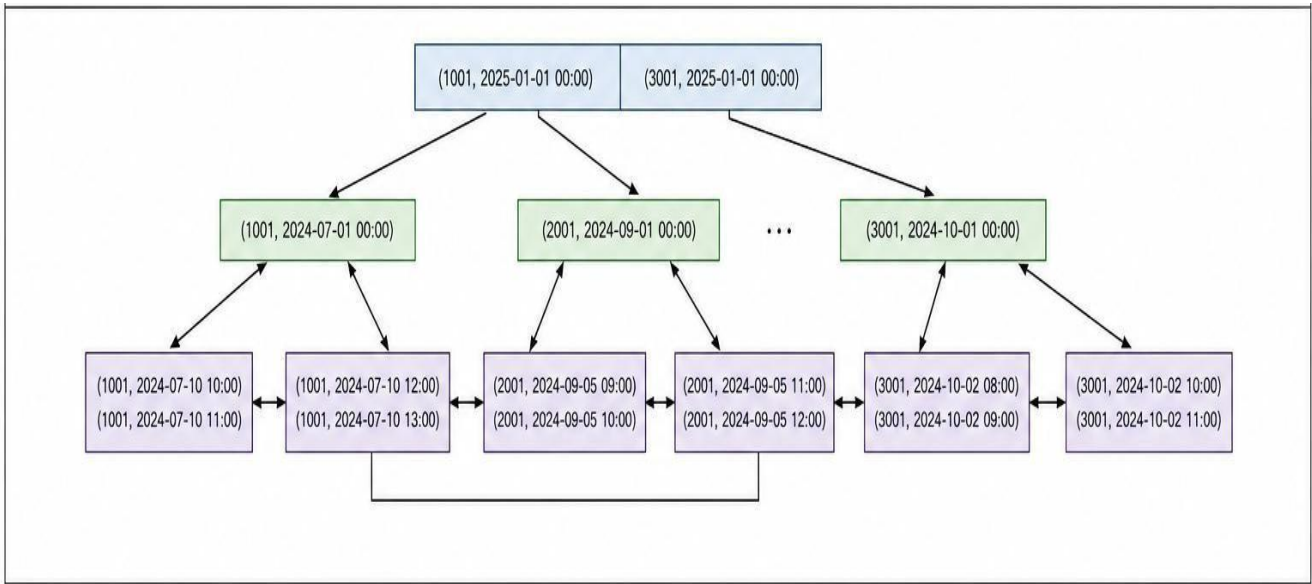


Figure 2: B+ Index Diagram

5.2 Secondary Index for Leak Detection

Leak detection commonly examines high consumption within a zone and time period. A secondary index on (ZoneID, ReadingTimestamp, Consumption) allows the DBMS to narrow the search to a zone/time range before applying the high-consumption condition.

5.3 Latest-Reading Strategy

For frequent latest-reading access, a separate current_meter_status/current_reading table is recommended. It contains one row per connection and is indexed or hashed by ConnectionID. This avoids repeatedly searching the historical billion-row table for MAX(timestamp). The historical table remains the source of record.

Index	Columns	Type	Purpose
PK / history index	ConnectionID, ReadingTimestamp	B+-tree	History and time-range retrieval
Leak index	ZoneID,ReadingTimestamp, Consumption	B+-tree	Zone/time anomaly search
Bill uniqueness	ConnectionID, BillingMonth	Unique B-tree	Duplicate-bill prevention
Bill account lookup	ConnectionID, Status	B-tree	Open-bill/payment operations
Payment lookup	BillID, PaymentTimestamp	B-tree	Reconciliation/audit

Complaint lookup	ConnectionID,Status,CreatedAt	B-tree	Complaint tracking
Current reading	ConnectionID	Hash/B-tree	Latest-reading point lookup

6. Hashing Scheme for Connection Lookup

Hashing is used for the exact-key workload: retrieving the latest reading for a known ConnectionID. The logical hash function is $h(\text{ConnectionID}) = \text{ConnectionID} \bmod B$, where B is the number of buckets. For example, with 10,000 buckets, ConnectionID 192471048 maps to bucket 1048.

Collision resolution uses separate chaining. Each bucket contains a chain/list of records or pointers to records. When two ConnectionIDs map to the same bucket, they are stored in the same chain and searched using the exact ConnectionID. This provides expected near-constant-time lookup when the load factor is controlled.

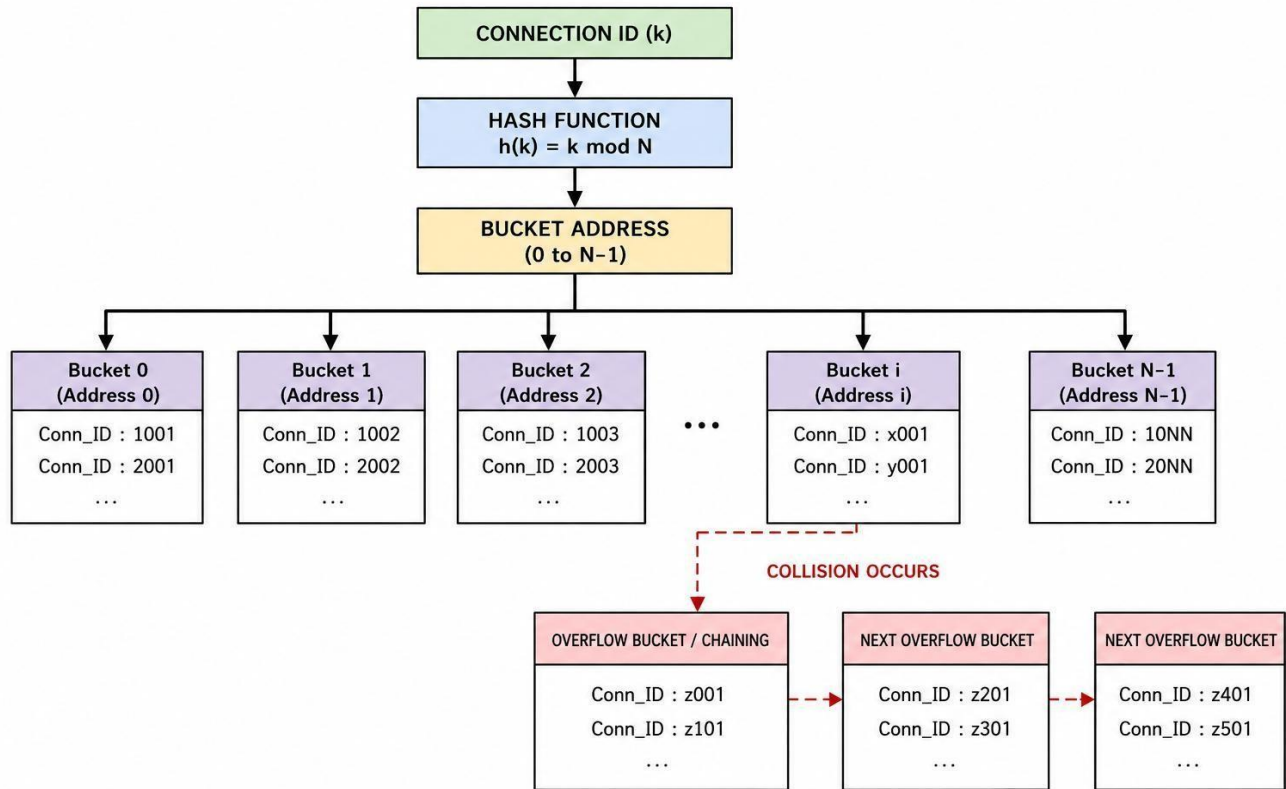


Figure 3: Hashing Diagram for Connection Lookup

6.1 Dynamic Growth

A static hash table is simple but becomes inefficient when the number of connections grows. For a production system, extendible hashing or another dynamic hashing strategy is preferable. Bucket splitting keeps the load factor controlled without requiring a complete redesign.

6.2 Hashing vs B+-Tree

Access Need	Hashing	B+-Tree	Selected
Exact ConnectionID lookup	Excellent	Very good	Hash
Connection history by time range	Poor	Excellent	B+-tree
Zone/time leak detection	Poor	Excellent	B+-tree
Ordered traversal	Poor	Excellent	B+-tree
Latest-reading point lookup	Excellent	Very good	Hash/current table

7. Storage and Retrieval Efficiency Analysis

The original flat sequential approach has approximately $O(N)$ point-lookup cost because records may need to be scanned. With a B+-tree, a connection/time query is approximately $O(\log_B N + k)$, where k is the number of qualifying records returned. A hash lookup has expected $O(1)$ access for an exact key, subject to a controlled load factor and collision chain length.

Operation	Flat Sequential File	Indexed/Hashed Design	Expected Effect
Latest reading for one connection	$O(N)$ scan	Expected $O(1)$ hash lookup	Very large improvement
6-month connection history	$O(N)$ scan	$O(\log N + k)$	Avoids full scan
Zone leak range	$O(N)$ scan	$O(\log N + k)$ index range scan	Scans relevant range
Monthly ingestion	Sequential write	Write + index maintenance	Higher write overhead
Storage	Data only	Data + index/hash overhead	Higher storage, justified by reads

For the given assumptions, hourly readings from 2 million connections produce approximately 48 million rows per day. At 30 days this is approximately 1.44 billion readings per month. This scale makes

repeated full scans operationally unsuitable for interactive history and anomaly queries. Indexes consume additional storage and add maintenance cost, so only workload-justified indexes are retained.

8. Relational Database and Schema Design

The transactional schema is normalized around connections, bills, payments, and complaints. The main integrity rules are primary keys, foreign keys, CHECK constraints where supported, NOT NULL requirements, and a unique constraint on connection plus billing month.

Table	Important Columns	Key / Constraint
Connection	ConnectionID, ZoneID, CustomerName, Status	PK ConnectionID
Zone	ZoneID, ZoneName	PK ZoneID
MeterReading	ReadingID, ConnectionID, ZoneID, ReadingTimestamp, Consumption	PK; FK ConnectionID; indexes
CurrentReading	ConnectionID, ReadingTimestamp, Consumption	PK ConnectionID
Bill	BillID, ConnectionID, BillingMonth, Units, Amount, DueAmount, Status	PK; FK; UNIQUE(ConnectionID,BillingMonth)
Payment	PaymentID, BillID, PaymentTimestamp, Amount, Mode	PK; FK BillID
Complaint	ComplaintID, ConnectionID, ComplaintType, Description, Status, CreatedAt, UpdatedAt	PK; FK ConnectionID
BillAudit	AuditID, BillID, Action, OperatorID, ActionTime	PK; audit trail

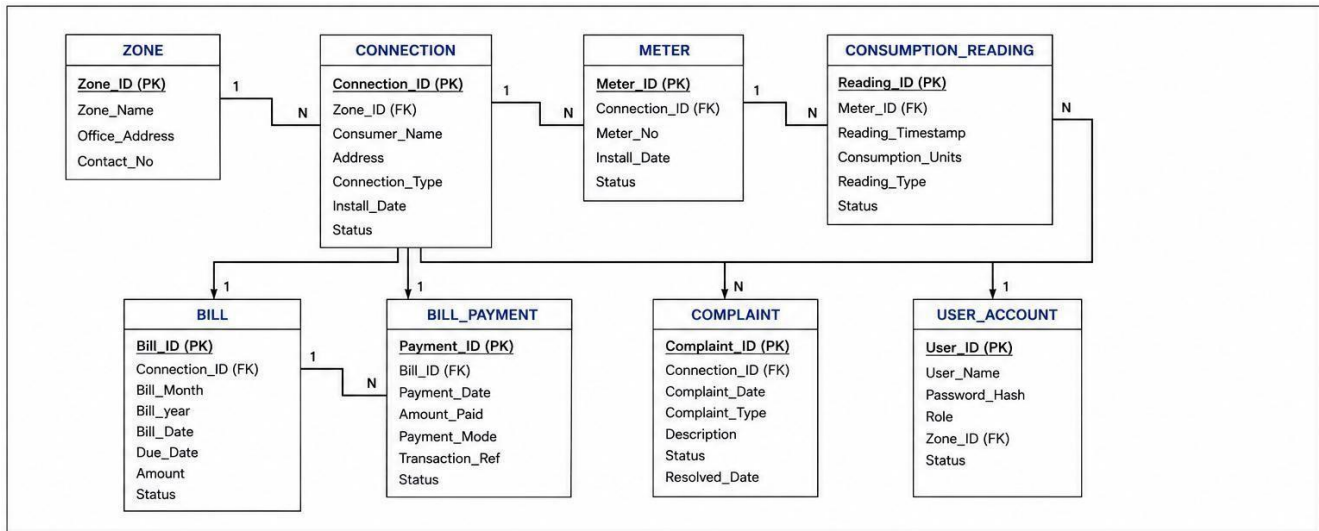


Figure 4: Relational schema of the Smart Water Utility Consumption, Billing, and Complaint Management System.

8.1 Integrity Rules

- One connection cannot have two bills for the same billing month.
- Every payment must reference an existing bill.
- Every complaint must reference an existing connection.
- Consumption must be non-negative.
- Payment amount must be positive.
- Bill due amount cannot become negative.
- Updates to monetary and status fields occur inside transactions.

9. SQL Query Design

9.1 Connection History

SELECT ReadingTimestamp, Consumption FROM MeterReading WHERE ConnectionID = ? AND ReadingTimestamp >= ? AND ReadingTimestamp < ? ORDER BY ReadingTimestamp;

9.2 Latest Reading

SELECT ReadingTimestamp, Consumption FROM CurrentReading WHERE ConnectionID = ?;

9.3 Leak-Detection Query

SELECT ConnectionID, ReadingTimestamp, Consumption FROM MeterReading WHERE ZoneID = ? AND ReadingTimestamp >= ? AND ReadingTimestamp < ? AND Consumption > ? ORDER BY Consumption DESC;

9.4 Monthly Bill Generation

```
SELECT ConnectionID, SUM(Consumption) AS UnitsConsumed FROM MeterReading WHERE ReadingTimestamp >= ? AND ReadingTimestamp < ? GROUP BY ConnectionID;
```

9.5 Payment Reconciliation

```
SELECT b.BillID, b.ConnectionID, b.Amount, COALESCE(SUM(p.Amount),0) AS Paid, b.Amount-COALESCE(SUM(p.Amount),0) AS Balance FROM Bill b LEFT JOIN Payment p ON p.BillID=b.BillID GROUP BY b.BillID, b.ConnectionID, b.Amount;
```

9.6 Complaint Status Report

```
SELECT c.Status, COUNT(*) AS ComplaintCount FROM Complaint c GROUP BY c.Status ORDER BY ComplaintCount DESC;
```

9.7 Subquery for Connections with Outstanding Bills

```
SELECT ConnectionID FROM Bill WHERE DueAmount > 0 AND ConnectionID IN (SELECT ConnectionID FROM Connection WHERE Status='ACTIVE');
```

10. Query Processing and Execution Plan Analysis

A DBMS processes SQL through parsing/validation, optimization, and execution. The optimizer chooses access paths such as index lookup, index range scan, table scan, join order, grouping method, and sorting strategy. EXPLAIN or EXPLAIN ANALYZE should be used to capture the chosen plan.

Query	Unoptimized Risk	Optimized Plan	Evidence to Capture
Connection history	Full table scan + sort	Composite B+-tree range scan	EXPLAIN output + timing
Leak detection	Full scan of readings	Zone/time index range scan	EXPLAIN output + rows examined
Payment reconciliation	Large join + unnecessary rows	Bill/payment indexed join	EXPLAIN + execution time
Complaint report	Repeated scan	Status index + aggregation	EXPLAIN + rows examined
Bill generation	Repeated per-connection scans	One grouped range query	EXPLAIN + timing

10.1 Example EXPLAIN Interpretation

A desirable plan for connection history should show an index/range access using `idx_meter_connection_time` rather than `ALL/table scan`. The estimated or actual rows examined should be substantially smaller than the complete `MeterReading` table. For the leak query, the preferred plan begins with `ZoneID` and `timestamp` restrictions using the secondary index. A plan with a full scan indicates that statistics, predicate structure, indexing, or data distribution should be investigated.

11. Query Optimization Implementation

- Create composite indexes matching the most selective and frequently used predicates.

- Rewrite repeated correlated work as grouped joins or pre-aggregated results when appropriate.
- Select only required columns instead of SELECT *.
- Use half-open time ranges ($\geq$ start and $<$ end) to avoid ambiguity at boundaries.
- Keep predicates sargable so indexes can be used.
- Maintain a CurrentReading table for frequent latest-reading requests.
- Use EXPLAIN/EXPLAIN ANALYZE before and after indexing.
- Compare rows examined, access type, estimated cost, and elapsed time.

12. Transaction Design and ACID

12.1 ACID Application

Property	Application
Atomicity	Bill/payment changes succeed together or are rolled back together
Consistency	Keys, uniqueness, foreign keys and balance rules remain valid
Isolation	Concurrent operators do not observe unsafe intermediate states
Durability	COMMIT makes completed bill/payment changes persistent

12.2 Bill Generation Transaction

7. BEGIN/START TRANSACTION.
8. Identify the target connection and billing month.
9. Lock/check the connection's billing state where required.
10. Insert the bill only if no bill exists for the cycle.
11. Rely on UNIQUE(ConnectionID, BillingMonth) as an integrity guard.
12. Use SAVEPOINT before optional audit/reporting work.
13. On success COMMIT; on failure ROLLBACK.

12.3 Payment Posting Transaction

14. BEGIN/START TRANSACTION.
15. SELECT the target bill row FOR UPDATE.
16. Validate that the bill is payable and the payment amount is valid.

17. Insert the payment record.
18. Update the bill due amount/status using the locked row.
19. Optionally create an audit entry.
20. COMMIT; on any error ROLLBACK.

12.4 SAVEPOINT Example

START TRANSACTION; ... SAVEPOINT before_audit; ... ROLLBACK TO SAVEPOINT before_audit; ... COMMIT;

The exact syntax varies slightly by DBMS. The submitted implementation should use the syntax of the selected DBMS.

13. Concurrency Control Analysis

Anomaly	Cause	Control
Duplicate bill	Two operators both see no existing bill	Unique constraint + transaction + row locking
Lost payment update	Two updates use the same stale due amount	SELECT ... FOR UPDATE and atomic update
Lost complaint update	Two operators overwrite same row	Row-level locking/version check
Dirty read	One transaction reads uncommitted data	READ COMMITTED or stronger
Non-repeatable read	Same row changes during transaction	REPEATABLE READ where required
Deadlock	Transactions lock resources in inconsistent order	Consistent lock order + retry

13.1 Selected Isolation Strategy

READ COMMITTED is recommended as the general operating isolation level because it avoids dirty reads while reducing unnecessary locking compared with SERIALIZABLE. Critical bill-generation and payment operations should use row-level locks such as SELECT ... FOR UPDATE where supported. The unique bill constraint provides an independent integrity barrier against duplicate billing. SERIALIZABLE can be used for narrowly scoped operations if testing shows that stronger isolation is required, but it may reduce throughput under peak load.

13.2 Concurrent Scenario

Operator A and Operator B attempt to generate a bill for the same connection and month. Both requests execute inside transactions. The system checks/locks the relevant state and the unique constraint permits only one bill. The losing transaction receives a duplicate-key/conflict condition and rolls back or treats the existing bill as the successful outcome. Thus two independent successful bill rows cannot be created for the same cycle.

14. Implementation Plan and Evidence

The recommended implementation platform is MySQL 8.x with MySQL Workbench, although PostgreSQL or Oracle may be substituted if the same required concepts are demonstrated.

14.1 Suggested MySQL Index Statements

```
CREATE INDEX idx_meter_connection_time ON MeterReading(ConnectionID, ReadingTimestamp);
```

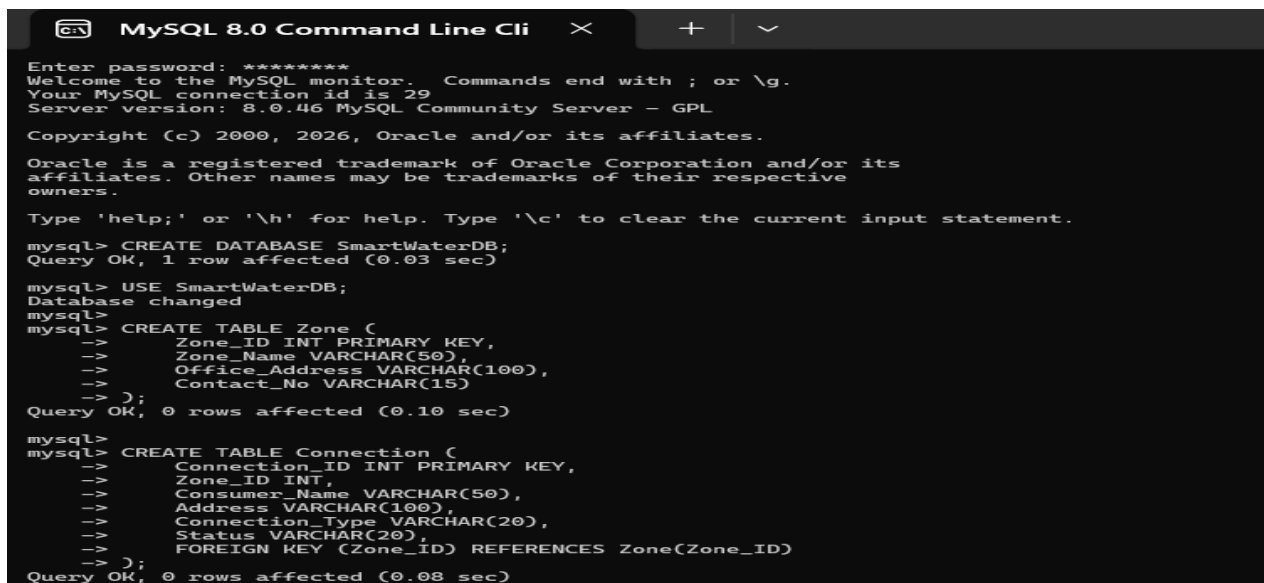
```
CREATE INDEX idx_meter_zone_time_consumption ON MeterReading(ZoneID, ReadingTimestamp, Consumption);
```

```
CREATE UNIQUE INDEX uq_bill_connection_month ON Bill(ConnectionID, BillingMonth);
```

```
CREATE INDEX idx_payment_bill_time ON Payment(BillID, PaymentTimestamp);
```

```
CREATE INDEX idx_complaint_conn_status ON Complaint(ConnectionID, Status, CreatedAt);
```

14.2 MySQL Implementation



```
MySQL 8.0 Command Line Cli
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 29
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE SmartWaterDB;
Query OK, 1 row affected (0.03 sec)

mysql> USE SmartWaterDB;
Database changed
mysql>
mysql> CREATE TABLE Zone (
  ->   Zone_ID INT PRIMARY KEY,
  ->   Zone_Name VARCHAR(50),
  ->   Office_Address VARCHAR(100),
  ->   Contact_No VARCHAR(15)
  -> );
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> CREATE TABLE Connection (
  ->   Connection_ID INT PRIMARY KEY,
  ->   Zone_ID INT,
  ->   Consumer_Name VARCHAR(50),
  ->   Address VARCHAR(100),
  ->   Connection_Type VARCHAR(20),
  ->   Status VARCHAR(20),
  ->   FOREIGN KEY (Zone_ID) REFERENCES Zone(Zone_ID)
  -> );
Query OK, 0 rows affected (0.08 sec)
```

```
mysql> CREATE TABLE Meter (
->   Meter_ID INT PRIMARY KEY,
->   Connection_ID INT,
->   Meter_No VARCHAR(30),
->   Install_Date DATE,
->   Status VARCHAR(20),
->   FOREIGN KEY (Connection_ID) REFERENCES Connection(Connection_ID)
-> );
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> CREATE TABLE Consumption_Reading (
->   Reading_ID INT PRIMARY KEY,
->   Meter_ID INT,
->   Reading_Timestamp DATETIME,
->   Consumption_Units DECIMAL(10,2),
->   Reading_Type VARCHAR(20),
->   Status VARCHAR(20),
->   FOREIGN KEY (Meter_ID) REFERENCES Meter(Meter_ID)
-> );
Query OK, 0 rows affected (0.07 sec)
```

```
MySQL 8.0 Command Line Cli  ×  +  ∨

mysql> CREATE TABLE Bill (
->   Bill_ID INT PRIMARY KEY,
->   Connection_ID INT,
->   Bill_Month INT,
->   Bill_Year INT,
->   Bill_Date DATE,
->   Due_Date DATE,
->   Amount DECIMAL(10,2),
->   Status VARCHAR(20),
->   FOREIGN KEY (Connection_ID) REFERENCES Connection(Connection_ID),
->   UNIQUE(Connection_ID, Bill_Month, Bill_Year)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE Bill_Payment (
->   Payment_ID INT PRIMARY KEY,
->   Bill_ID INT,
->   Payment_Date DATE,
->   Amount_Paid DECIMAL(10,2),
->   Payment_Mode VARCHAR(20),
->   Transaction_Ref VARCHAR(50),
->   Status VARCHAR(20),
->   FOREIGN KEY (Bill_ID) REFERENCES Bill(Bill_ID)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE Complaint (
->   Complaint_ID INT PRIMARY KEY,
->   Connection_ID INT,
->   Complaint_Date DATE,
->   Complaint_Type VARCHAR(50),
->   Description VARCHAR(200),
->   Status VARCHAR(20),
->   Resolved_Date DATE,
->   FOREIGN KEY (Connection_ID) REFERENCES Connection(Connection_ID)
-> );
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> INSERT INTO Zone VALUES
-> (1,'North Zone','Chennai North','9000000001'),
-> (2,'South Zone','Chennai South','9000000002'),
-> (3,'Central Zone','Chennai Central','9000000003');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Connection VALUES
-> (1001,1,'Ravi','Anna Nagar','Domestic','Active'),
-> (1002,1,'Priya','Shenoy Nagar','Domestic','Active'),
-> (2001,2,'Kumar','Velachery','Commercial','Active'),
-> (3001,3,'Anitha','T Nagar','Domestic','Active');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Meter VALUES
-> (501,1001,'MTR1001','2024-01-10','Active'),
-> (502,1002,'MTR1002','2024-02-15','Active'),
-> (503,2001,'MTR2001','2024-03-20','Active'),
-> (504,3001,'MTR3001','2024-04-10','Active');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```

MySQL 8.0 Command Line Cli
mysql> INSERT INTO Consumption_Reading VALUES
-> (1,501,'2026-08-01 10:00:00',25.50,'Hourly','Valid'),
-> (2,501,'2026-08-01 11:00:00',28.20,'Hourly','Valid'),
-> (3,502,'2026-08-01 10:00:00',18.40,'Hourly','Valid'),
-> (4,503,'2026-08-01 10:00:00',45.60,'Hourly','Valid'),
-> (5,504,'2026-08-01 10:00:00',22.30,'Hourly','Valid');
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Bill VALUES
-> (101,1001,8,2026,'2026-08-01','2026-08-15',850.00,'Pending'),
-> (102,1002,8,2026,'2026-08-01','2026-08-15',620.00,'Pending'),
-> (103,2001,8,2026,'2026-08-01','2026-08-15',1500.00,'Pending'),
-> (104,3001,8,2026,'2026-08-01','2026-08-15',730.00,'Pending');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Bill_Payment VALUES
-> (1,101,'2026-08-05',500.00,'UPI','TXN001','Success'),
-> (2,102,'2026-08-06',620.00,'Card','TXN002','Success');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO Complaint VALUES
-> (1,1001,'2026-08-02','Leakage','Possible pipeline leakage','Open',NULL),
-> (2,2001,'2026-08-03','Meter','Meter reading abnormal','Resolved','2026-08-05');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> SHOW TABLES;
+-----+
| Tables_in_smartwaterdb |
+-----+
| bill                     |
| bill_payment            |
| complaint               |
| connection              |
| consumption_reading     |
| meter                   |
| zone                    |
+-----+
7 rows in set (0.01 sec)

mysql>
mysql> SELECT * FROM Zone;
+-----+-----+-----+-----+
| Zone_ID | Zone_Name | Office_Address | Contact_No |
+-----+-----+-----+-----+
| 1       | North Zone | Chennai North  | 9000000001 |
| 2       | South Zone | Chennai South  | 9000000002 |
| 3       | Central Zone | Chennai Central | 9000000003 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

MySQL 8.0 Command Line Cli
mysql> SELECT * FROM Connection;
+-----+-----+-----+-----+-----+-----+
| Connection_ID | Zone_ID | Consumer_Name | Address       | Connection_Type | Status |
+-----+-----+-----+-----+-----+-----+
| 1001          | 1       | Ravi          | Anna Nagar   | Domestic        | Active |
| 1002          | 1       | Priya         | Shenoy Nagar | Domestic        | Active |
| 2001          | 2       | Kumar        | Velachery    | Commercial      | Active |
| 3001          | 3       | Anitha       | T Nagar      | Domestic        | Active |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Meter;
+-----+-----+-----+-----+-----+
| Meter_ID | Connection_ID | Meter_No | Install_Date | Status |
+-----+-----+-----+-----+-----+
| 501      | 1001          | MTR1001 | 2024-01-10   | Active |
| 502      | 1002          | MTR1002 | 2024-02-15   | Active |
| 503      | 2001          | MTR2001 | 2024-03-20   | Active |
| 504      | 3001          | MTR3001 | 2024-04-10   | Active |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Consumption_Reading;
+-----+-----+-----+-----+-----+-----+-----+
| Reading_ID | Meter_ID | Reading_Timestamp | Consumption_Units | Reading_Type | Status |
+-----+-----+-----+-----+-----+-----+-----+
| 1          | 501      | 2026-08-01 10:00:00 | 25.50            | Hourly       | Valid  |
| 2          | 501      | 2026-08-01 11:00:00 | 28.20            | Hourly       | Valid  |
| 3          | 502      | 2026-08-01 10:00:00 | 18.40            | Hourly       | Valid  |
| 4          | 503      | 2026-08-01 10:00:00 | 45.60            | Hourly       | Valid  |
| 5          | 504      | 2026-08-01 10:00:00 | 22.30            | Hourly       | Valid  |
+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Bill;
+-----+-----+-----+-----+-----+-----+-----+
| Bill_ID | Connection_ID | Bill_Month | Bill_Year | Bill_Date | Due_Date | Amount | Status |
+-----+-----+-----+-----+-----+-----+-----+
| 101     | 1001          | 8          | 2026     | 2026-08-01 | 2026-08-15 | 850.00 | Pending |
| 102     | 1002          | 8          | 2026     | 2026-08-01 | 2026-08-15 | 620.00 | Pending |
| 103     | 2001          | 8          | 2026     | 2026-08-01 | 2026-08-15 | 1500.00 | Pending |
| 104     | 3001          | 8          | 2026     | 2026-08-01 | 2026-08-15 | 730.00 | Pending |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM Bill_Payment;
+-----+-----+-----+-----+-----+-----+-----+
| Payment_ID | Bill_ID | Payment_Date | Amount_Paid | Payment_Mode | Transaction_Ref | Status |
+-----+-----+-----+-----+-----+-----+-----+
| 1          | 101     | 2026-08-05   | 500.00      | UPI          | TXN001          | Success |
| 2          | 102     | 2026-08-06   | 620.00      | Card         | TXN002          | Success |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

```

MySQL 8.0 Command Line Cli
mysql> SELECT * FROM Complaint;
+-----+-----+-----+-----+-----+-----+-----+
| Complaint_ID | Connection_ID | Complaint_Date | Complaint_Type | Description | Status | Resolved_Date |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1001 | 2026-08-02 | Leakage | Possible pipeline leakage | Open | NULL |
| 2 | 2001 | 2026-08-03 | Meter | Meter reading abnormal | Resolved | 2026-08-05 |
+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT *
-> FROM Consumption_Reading
-> WHERE Meter_ID = 501
-> ORDER BY Reading_Timestamp DESC
-> LIMIT 1;
+-----+-----+-----+-----+-----+-----+
| Reading_ID | Meter_ID | Reading_Timestamp | Consumption_Units | Reading_Type | Status |
+-----+-----+-----+-----+-----+-----+
| 2 | 501 | 2026-08-01 11:00:00 | 28.20 | Hourly | Valid |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT m.Connection_ID,
-> r.Reading_Timestamp,
-> r.Consumption_Units
-> FROM Meter m
-> JOIN Consumption_Reading r
-> ON m.Meter_ID = r.Meter_ID
-> WHERE m.Connection_ID = 1001
-> ORDER BY r.Reading_Timestamp;
+-----+-----+-----+
| Connection_ID | Reading_Timestamp | Consumption_Units |
+-----+-----+-----+
| 1001 | 2026-08-01 10:00:00 | 25.50 |
| 1001 | 2026-08-01 11:00:00 | 28.20 |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT m.Connection_ID,
-> r.Reading_Timestamp,
-> r.Consumption_Units
-> FROM Meter m
-> JOIN Consumption_Reading r
-> ON m.Meter_ID = r.Meter_ID
-> WHERE r.Consumption_Units > 30;
+-----+-----+-----+
| Connection_ID | Reading_Timestamp | Consumption_Units |
+-----+-----+-----+
| 2001 | 2026-08-01 10:00:00 | 45.60 |
+-----+-----+-----+
1 row in set (0.00 sec)

mysql> CREATE INDEX idx_consumption_meter_time
-> ON Consumption_Reading(Meter_ID, Reading_Timestamp);
Query OK, 0 rows affected (0.10 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_bill_connection
-> ON Bill(Connection_ID);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

```

```

MySQL 8.0 Command Line Cli
mysql> SHOW INDEX FROM Consumption_Reading;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| consumption_reading | 0 | PRIMARY | 1 | Reading_ID | A | 5 | NULL | NULL | YES | BTREE | | | YES | NULL |
| consumption_reading | 1 | idx_consumption_meter_time | 1 | Meter_ID | A | 4 | NULL | NULL | YES | BTREE | | | YES | NULL |
| consumption_reading | 1 | idx_consumption_meter_time | 2 | Reading_Timestamp | A | 5 | NULL | NULL | YES | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.02 sec)

mysql> SHOW INDEX FROM Bill;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| bill | 0 | PRIMARY | 1 | Bill_ID | A | 4 | NULL | NULL | YES | BTREE | | | YES | NULL |
| bill | 0 | Connection_ID | 1 | Connection_ID | A | 4 | NULL | NULL | YES | BTREE | | | YES | NULL |
| bill | 0 | Connection_ID | 2 | Bill_Month | A | 4 | NULL | NULL | YES | BTREE | | | YES | NULL |
| bill | 0 | Connection_ID | 3 | Bill_Year | A | 4 | NULL | NULL | YES | BTREE | | | YES | NULL |
| bill | 1 | idx_bill_connection | 1 | Connection_ID | A | 4 | NULL | NULL | YES | BTREE | | | YES | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.02 sec)

mysql> ALTER TABLE Connection
-> ADD COLUMN Hash_Bucket INT;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> UPDATE Connection
-> SET Hash_Bucket = MOD(Connection_ID, 4);
Query OK, 4 rows affected (0.01 sec)
Rows matched: 4 Changed: 4 Warnings: 0

mysql> SELECT Connection_ID,
-> Consumer_Name,
-> Hash_Bucket
-> FROM Connection
-> ORDER BY Hash_Bucket;
+-----+-----+-----+
| Connection_ID | Consumer_Name | Hash_Bucket |
+-----+-----+-----+
| 1001 | Ravi | 1 |
| 2001 | Kumar | 1 |
| 3001 | Anitha | 1 |
| 1002 | Priya | 2 |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> EXPLAIN
-> SELECT *
-> FROM Consumption_Reading
-> WHERE Meter_ID = 501;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | Table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | Consumption_Reading | NULL | ref | idx_consumption_meter_time | idx_consumption_meter_time | 5 | const | 2 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set, 1 warning (0.00 sec)

```

```

MySQL 8.0 Command Line Cli
mysql> SELECT c.Connection_ID,
-> c.Consumer_Name,
-> b.Bill_ID,
-> b.Amount,
-> b.Status
-> FROM Connection c
-> JOIN Bill b
-> ON c.Connection_ID = b.Connection_ID;
+-----+-----+-----+-----+-----+
| Connection_ID | Consumer_Name | Bill_ID | Amount | Status |
+-----+-----+-----+-----+-----+
| 1001 | Ravi | 101 | 850.00 | Pending |
| 1002 | Priya | 102 | 620.00 | Pending |
| 2001 | Kumar | 103 | 1500.00 | Pending |
| 3001 | Anitha | 104 | 730.00 | Pending |
+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT b.Bill_ID,
-> b.Amount,
-> COALESCE(SUM(p.Amount_Paid),0) AS Amount_Paid,
-> b.Amount - COALESCE(SUM(p.Amount_Paid),0) AS Balance
-> FROM Bill b
-> LEFT JOIN Bill_Payment p
-> ON b.Bill_ID = p.Bill_ID
-> GROUP BY b.Bill_ID, b.Amount;
+-----+-----+-----+-----+
| Bill_ID | Amount | Amount_Paid | Balance |
+-----+-----+-----+-----+
| 101 | 850.00 | 500.00 | 350.00 |
| 102 | 620.00 | 620.00 | 0.00 |
| 103 | 1500.00 | 0.00 | 1500.00 |
| 104 | 730.00 | 0.00 | 730.00 |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO Bill_Payment
-> VALUES
-> (3,103,CURDATE(),500.00,'UPI','TXN003','Success');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> UPDATE Bill
-> SET Status = 'Partially Paid'
-> WHERE Bill_ID = 103;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

```

```

MySQL 8.0 Command Line Cli
mysql> SELECT *
-> FROM Bill_Payment
-> WHERE Bill_ID = 103;
+-----+-----+-----+-----+-----+-----+-----+
| Payment_ID | Bill_ID | Payment_Date | Amount_Paid | Payment_Mode | Transaction_Ref | Status |
+-----+-----+-----+-----+-----+-----+-----+
| 3 | 103 | 2026-09-02 | 500.00 | UPI | TXN003 | Success |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT *
-> FROM Bill
-> WHERE Bill_ID = 103;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Bill_ID | Connection_ID | Bill_Month | Bill_Year | Bill_Date | Due_Date | Amount | Status |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 103 | 2001 | 8 | 2026 | 2026-08-01 | 2026-08-15 | 1500.00 | Partially Paid |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE Bill
-> SET Amount = 2000
-> WHERE Bill_ID = 104;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT *
-> FROM Bill
-> WHERE Bill_ID = 104;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Bill_ID | Connection_ID | Bill_Month | Bill_Year | Bill_Date | Due_Date | Amount | Status |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 104 | 3001 | 8 | 2026 | 2026-08-01 | 2026-08-15 | 730.00 | Pending |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE Bill
-> SET Status = 'Processing'
-> WHERE Bill_ID = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> SAVEPOINT bill_update;
Query OK, 0 rows affected (0.00 sec)

```

```

MySQL 8.0 Command Line Cli
mysql> UPDATE Bill
-> SET Amount = 9999
-> WHERE Bill_ID = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql>
mysql> ROLLBACK TO bill_update;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT *
-> FROM Bill
-> WHERE Bill_ID = 101;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Bill_ID | Connection_ID | Bill_Month | Bill_Year | Bill_Date | Due_Date | Amount | Status |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 101 | 1001 | 8 | 2026 | 2026-08-01 | 2026-08-15 | 850.00 | Processing |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql>
mysql> SELECT *
-> FROM Complaint;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Complaint_ID | Connection_ID | Complaint_Date | Complaint_Type | Description | Status | Resolved_Date |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1001 | 2026-08-02 | Leakage | Possible pipeline leakage | Open | NULL |
| 2 | 2001 | 2026-08-03 | Meter | Meter reading abnormal | Resolved | 2026-08-05 |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> SELECT *
-> FROM Complaint
-> WHERE Status = 'Open';
+-----+-----+-----+-----+-----+-----+-----+-----+
| Complaint_ID | Connection_ID | Complaint_Date | Complaint_Type | Description | Status | Resolved_Date |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | 1001 | 2026-08-02 | Leakage | Possible pipeline leakage | Open | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> |

```

14.3 Website Implementation

INSTRUCTIONS TO RUN THE PROJECT

1. Save this file as code.html.
2. Open the file location.
3. Double-click code.html.
4. The project will open in Google Chrome or any web browser.
5. No compilation or installation is required.
6. If the file does not open in a browser, right-click the file and select "Open with → Google Chrome".

FILE NAME: code.html

INPUT:

1. Connection ID and Consumer Name
 - Entered when adding a new water connection.
2. Zone and Connection Type
 - Select the consumer's zone and type.
3. Meter Reading Data
 - Connection ID, Meter ID, timestamp and consumption units are displayed/stored.

4. Payment Details
 - Enter Bill ID, payment amount and payment mode (UPI, Card, Cash or Net Banking).
5. Complaint Details
 - Enter Connection ID, complaint type and complaint description.
6. Search Input
 - Enter Connection ID or Consumer Name to search the respective records.

OUTPUT:

1. Dashboard
 - Displays total connections, meter readings, pending bills and open complaints.
2. Connection Management
 - Displays all registered consumers and their connection details.
3. Meter Readings
 - Displays meter readings and consumption units.
4. Billing Management
 - Displays bill ID, connection ID, amount, due date and payment status.
5. Payment Management
 - Displays successfully recorded payments.
6. Complaint Management
 - Displays registered complaints and their status.
7. Leak Detection
 - Detects high water consumption above the threshold and displays a warning alert.
8. Search Results
 - Displays only the records matching the entered search text.

CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

<title>Smart Water Utility Management System</title>

<style>

```
*{
  box-sizing:border-box;
  margin:0;
  padding:0;
  font-family:Arial, sans-serif;
}
```

```
body{
  background:linear-gradient(135deg,#e8f8ff,#f7f4ff);
  color:#222;
}
```

/* HEADER */

```
header{
  background:linear-gradient(135deg,#0077b6,#00b4d8,#48cae4);
  color:white;
  padding:16px 30px;
  display:flex;
  justify-content:space-between;
  align-items:center;
  box-shadow:0 4px 12px rgba(0,0,0,.15);
}
```

```
header h1 {
  font-size:23px;
}
```

```
header p{
  font-size:13px;
  margin-top:5px;
}
```

/* ADMIN PROFILE */

```
.admin-profile{
  display:flex;
  align-items:center;
  gap:10px;
  background:rgba(255,255,255,.18);
  padding:7px 14px 7px 7px;
  border-radius:30px;
}
```

```
.profile-img{
  width:42px;
```



```

height:42px;
border-radius:50%;
background:white;
display:flex;
align-items:center;
justify-content:center;
font-size:25px;
box-shadow:0 2px 6px rgba(0,0,0,.2);
}

.admin-text{
  font-weight:bold;
  font-size:15px;
}

/* MAIN CONTAINER */

.container{
  display:flex;
  min-height:calc(100vh - 70px);
}

/* SIDEBAR */

.sidebar{
  width:235px;
  background:linear-gradient(180deg,#ffffff,#eefaff);
  border-right:1px solid #d5e8ef;
  padding-top:20px;
  box-shadow:3px 0 10px rgba(0,0,0,.05);
}

.sidebar button{
  width:100%;
  border:none;
  background:transparent;
  text-align:left;
  padding:15px 25px;
  font-size:15px;
  cursor:pointer;
  color:#164e63;
  transition:.3s;
}

.sidebar button:hover{
  background:#dff7ff;
  padding-left:32px;
  color:#0077b6;
}

```

```

.sidebar button.active{
  background:linear-gradient(90deg,#0077b6,#00b4d8);
  color:white;
  border-radius:0 25px 25px 0;
  box-shadow:0 4px 10px rgba(0,119,182,.25);
}

/* MAIN */

.main{
  flex:1;
  padding:28px;
}

.page{
  display:none;
}

.page.active{
  display:block;
  animation:fadeIn .3s ease;
}

@keyframes fadeIn{
  from{
    opacity:0;
    transform:translateY(8px);
  }
  to{
    opacity:1;
    transform:translateY(0);
  }
}

.page h2{
  color:#0077b6;
  margin-bottom:20px;
  font-size:25px;
}

/* DASHBOARD CARDS */

.cards{
  display:grid;
  grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
  gap:20px;
  margin-bottom:30px;
}

.card{

```

```

background:white;
padding:20px;
border-radius:15px;
box-shadow:0 4px 12px rgba(0,0,0,.09);
border-top:5px solid #00b4d8;
transition:.3s;
}

.card:hover{
transform:translateY(-5px);
box-shadow:0 8px 18px rgba(0,0,0,.13);
}

.card h3{
font-size:14px;
color:#666;
margin-bottom:10px;
}

.card .number{
font-size:29px;
color:#0077b6;
font-weight:bold;
}

.card small{
color:#777;
}

/* DIFFERENT CARD COLORS */

.cards .card:nth-child(1){
border-top-color:#0077b6;
}

.cards .card:nth-child(2){
border-top-color:#00b4d8;
}

.cards .card:nth-child(3){
border-top-color:#f4a261;
}

.cards .card:nth-child(4){
border-top-color:#e63946;
}

/* TABLE */

.table-box{

```

```

background:white;
border-radius:15px;
box-shadow:0 4px 12px rgba(0,0,0,.08);
padding:20px;
overflow-x:auto;
}

.table-box h3{
margin-bottom:15px;
color:#164e63;
}

table{
width:100%;
border-collapse:collapse;
}

th{
background:linear-gradient(90deg,#0077b6,#00b4d8);
color:white;
padding:13px;
text-align:left;
}

td{
padding:12px;
border-bottom:1px solid #e1edf1;
}

tr:hover{
background:#f0fbff;
}

/* SEARCH */

.search{
margin-bottom:15px;
display:flex;
gap:10px;
}

.search input,
.search select{
padding:11px;
border:1px solid #b8d9e5;
border-radius:7px;
outline:none;
}

.search input:focus{

```

```

border-color:#00b4d8;
box-shadow:0 0 5px rgba(0,180,216,.3);
}

.search input{
width:260px;
}

/* BUTTON */

.primary-btn{
background:linear-gradient(90deg,#0077b6,#00b4d8);
color:white;
border:none;
padding:11px 18px;
border-radius:7px;
cursor:pointer;
font-weight:bold;
transition:.3s;
}

.primary-btn:hover{
background:linear-gradient(90deg,#005f8f,#0096c7);
transform:translateY(-2px);
box-shadow:0 4px 10px rgba(0,119,182,.25);
}

/* FORM */

.form-box{
background:white;
padding:22px;
border-radius:15px;
box-shadow:0 4px 12px rgba(0,0,0,.08);
margin-bottom:25px;
border-left:5px solid #00b4d8;
}

.form-box h3{
color:#164e63;
}

.form-grid{
display:grid;
grid-template-columns:repeat(auto-fit,minmax(200px,1fr));
gap:15px;
}

.form-grid input,
.form-grid select,

```

```

.form-grid textarea{
  width:100%;
  padding:1 1px;
  border:1px solid #b8d9e5;
  border-radius:7px;
  outline:none;
}

.form-grid input:focus,
.form-grid select:focus,
.form-grid textarea:focus{
  border-color:#00b4d8;
  box-shadow:0 0 5px rgba(0,180,216,.25);
}

.form-grid textarea{
  min-height:80px;
}

/* STATUS */

.status{
  padding:6px 11px;
  border-radius:15px;
  font-size:12px;
  font-weight:bold;
}

.active-status{
  background:#d9f6e5;
  color:#147239;
}

.pending{
  background:#fff0c2;
  color:#8a6500;
}

.paid{
  background:#d9f6e5;
  color:#147239;
}

.open{
  background:#ffd9d9;
  color:#a30000;
}

.resolved{
  background:#d9f6e5;

```

```

    color:#147239;
}

/* LEAK ALERT */

.alert-box{
    background:white;
    border-left:6px solid #e63946;
    padding:20px 25px;
    margin-bottom:12px;
    border-radius:10px;
    box-shadow:0 4px 10px rgba(0,0,0,.07);
    max-width:650px;
}

.alert-title{
    color:#d62828;
    font-size:18px;
    font-weight:bold;
    margin-bottom:18px;
    padding-bottom:12px;
    border-bottom:1px solid #e6e6e6;
}

.alert-details{
    display:flex;
    flex-direction:column;
    gap:8px;
}

.detail-row{
    display:grid;
    grid-template-columns:160px 1fr;
    align-items:center;
    padding:8px 0;
}

.detail-label{
    color:#666;
    font-weight:bold;
}

.detail-value{
    color:#222;
    font-weight:bold;
}

/* FOOTER */

footer{

```

```

text-align:center;
background:linear-gradient(90deg,#0077b6,#00b4d8);
color:white;
padding:13px;
font-size:13px;
}

```

```

/* MOBILE */

```

```

@media(max-width:800px){

```

```

.container{
display:block;
}

```

```

.sidebar{
width:100%;
display:flex;
overflow-x:auto;
}

```

```

.sidebar button{
min-width:150px;
text-align:center;
}

```

```

header h1 {
font-size:17px;
}

```

```

.admin-text{
display:none;
}

```

```

.detail-row{
grid-template-columns:130px 1fr;
}

```

```

}

```

```

</style>

```

```

</head>

```

```

<body>

```

```

<!-- HEADER -->

```

```

<header>

```

```

<div>

```



```
<h1> Smart Water Utility Management System</h1>
<p>Consumption • Billing • Payment • Complaint Management</p>
</div>
```

```
<div class="admin-profile">
```

```
<div class="profile-img"> </div>
```

```
<div class="admin-text">
Admin
</div>
```

```
</div>
```

```
</header>
```

```
<!-- MAIN CONTAINER -->
```

```
<div class="container">
```

```
<!-- SIDEBAR -->
```

```
<div class="sidebar">
```

```
<button class="nav active"
onclick="showPage('dashboard',this)">
 Dashboard
</button>
```

```
<button class="nav"
onclick="showPage('connections',this)">
 Connections
</button>
```

```
<button class="nav"
onclick="showPage('readings',this)">
Meter Readings
</button>
```

```
<button class="nav"
onclick="showPage('bills',this)">
 Bills
</button>
```

```
<button class="nav"
onclick="showPage('payments',this)">
 Payments
</button>
```

```

<button class="nav"
onclick="showPage('complaints',this)">
 Complaints
</button>

<button class="nav"
onclick="showPage('leaks',this)">
  ■ Leak Detection
</button>

</div>

<!-- MAIN CONTENT -->

<div class="main">

<!-- DASHBOARD -->

<section id="dashboard" class="page active">

<h2>  Dashboard</h2>

<div class="cards">

<div class="card">

<h3>Total Connections</h3>

<div class="number" id="connectionCount">
4
</div>

<small>Active consumers</small>

</div>

<div class="card">

<h3>Meter Readings</h3>

<div class="number" id="readingCount">
5
</div>

<small>Stored readings</small>

</div>

<div class="card">

```

<h3>Pending Bills</h3>

<div class="number">

3

</div>

<small>Awaiting payment</small>

</div>

<div class="card">

<h3>Open Complaints</h3>

<div class="number" id="complaintCount">

1

</div>

<small>Requires attention</small>

</div>

</div>

<div class="table-box">

<h3>Recent Meter Readings</h3>

<table>

<thead>

<tr>

<th>Reading ID</th>

<th>Connection</th>

<th>Time</th>

<th>Units</th>

<th>Status</th>

</tr>

</thead>

<tbody>

<tr>

<td>1</td>

<td>1001</td>

<td>01-Aug-2026 10:00</td>

<td>25.50</td>

<td>

```
<span class="status active-status">
Valid
</span>
</td>
</tr>
```

```
<tr>
<td>2</td>
<td>1001</td>
<td>01-Aug-2026 11:00</td>
<td>28.20</td>
<td>
<span class="status active-status">
Valid
</span>
</td>
</tr>
```

```
<tr>
<td>3</td>
<td>1002</td>
<td>01-Aug-2026 10:00</td>
<td>18.40</td>
<td>
<span class="status active-status">
Valid
</span>
</td>
</tr>
```

```
<tr>
<td>4</td>
<td>2001</td>
<td>01-Aug-2026 10:00</td>
<td>45.60</td>
<td>
<span class="status active-status">
Valid
</span>
</td>
</tr>
```

```
</tbody>
```

```
</table>
```

```
</div>
```

```
</section>
```

```

<!-- CONNECTIONS -->

<section id="connections" class="page">

<h2>3 Connection Management</h2>

<div class="form-box">

<h3>Add New Connection</h3>

<br>

<div class="form-grid">

<input
id="connId"
placeholder="Connection ID">

<input
id="consumerName"
placeholder="Consumer Name">

<select id="zone">

<option>North Zone</option>
<option>South Zone</option>
<option>Central Zone</option>

</select>

<select id="connectionType">

<option>Domestic</option>
<option>Commercial</option>

</select>

</div>

<br>

<button
class="primary-btn"
onclick="addConnection()">

+ Add Connection

</button>

</div>

```

```

<div class="table-box">

<div class="search">

<input
id="connectionSearch"
onkeyup="searchConnections()"
placeholder="Search connection or consumer">

</div>

<table id="connectionTable">

<thead>

<tr>
<th>Connection ID</th>
<th>Consumer</th>
<th>Zone</th>
<th>Type</th>
<th>Status</th>
</tr>

</thead>

<tbody>

<tr>
<td>1001</td>
<td>Ravi</td>
<td>North Zone</td>
<td>Domestic</td>
<td>
<span class="status active-status">
Active
</span>
</td>
</tr>

<tr>
<td>1002</td>
<td>Priya</td>
<td>North Zone</td>
<td>Domestic</td>
<td>
<span class="status active-status">
Active
</span>
</td>
</tr>

```

```

</tr>

<tr>
<td>2001</td>
<td>Kumar</td>
<td>South Zone</td>
<td>Commercial</td>
<td>
<span class="status active-status">
Active
</span>
</td>
</tr>

<tr>
<td>3001</td>
<td>Anitha</td>
<td>Central Zone</td>
<td>Domestic</td>
<td>
<span class="status active-status">
Active
</span>
</td>
</tr>

</tbody>

</table>

</div>

</section>

<!-- METER READINGS -->

<section id="readings" class="page">

<h2>    Meter Consumption Readings</h2>

<div class="table-box">

<div class="search">

<input
id="readingSearch"
placeholder="Enter connection ID"
onkeyup="searchReadings()">

</div>

```

```

<table id="readingTable">

<thead>

<tr>
<th>Reading ID</th>
<th>Connection ID</th>
<th>Meter ID</th>
<th>Timestamp</th>
<th>Consumption Units</th>
</tr>

</thead>

<tbody>

<tr>
<td>1</td>
<td>1001</td>
<td>501</td>
<td>01-Aug-2026 10:00</td>
<td>25.50</td>
</tr>

<tr>
<td>2</td>
<td>1001</td>
<td>501</td>
<td>01-Aug-2026 11:00</td>
<td>28.20</td>
</tr>

<tr>
<td>3</td>
<td>1002</td>
<td>502</td>
<td>01-Aug-2026 10:00</td>
<td>18.40</td>
</tr>

<tr>
<td>4</td>
<td>2001</td>
<td>503</td>
<td>01-Aug-2026 10:00</td>
<td>45.60</td>
</tr>

<tr>

```



```

<td>5</td>
<td>3001</td>
<td>504</td>
<td>01-Aug-2026 10:00</td>
<td>22.30</td>
</tr>

</tbody>

</table>

</div>

</section>

<!-- BILLS -->

<section id="bills" class="page">

<h2>> Billing Management</h2>

<div class="table-box">

<table>

<thead>

<tr>
<th>Bill ID</th>
<th>Connection ID</th>
<th>Month</th>
<th>Amount</th>
<th>Due Date</th>
<th>Status</th>
</tr>

</thead>

<tbody>

<tr>
<td>101</td>
<td>1001</td>
<td>August 2026</td>
<td>₹850</td>
<td>15-Aug-2026</td>
<td>
<span class="status pending">
Pending
</span>

```

</td>

</tr>

<tr>

<td>102</td>

<td>1002</td>

<td>August 2026</td>

<td>₹620</td>

<td>15-Aug-2026</td>

<td>

Paid

</td>

</tr>

<tr>

<td>103</td>

<td>2001</td>

<td>August 2026</td>

<td>₹1500</td>

<td>15-Aug-2026</td>

<td>

Pending

</td>

</tr>

<tr>

<td>104</td>

<td>3001</td>

<td>August 2026</td>

<td>₹730</td>

<td>15-Aug-2026</td>

<td>

Pending

</td>

</tr>

</tbody>

</table>

</div>

</section>

```

<!-- PAYMENTS -->

<section id="payments" class="page">

<h2>  Payment Management</h2>

<div class="form-box">

<h3>Record Payment</h3>

<br>

<div class="form-grid">

<input
id="paymentBill"
placeholder="Bill ID">

<input
id="paymentAmount"
type="number"
placeholder="Amount">

<select id="paymentMode">

<option>UPI</option>
<option>Card</option>
<option>Cash</option>
<option>Net Banking</option>

</select>

</div>

<br>

<button
class="primary-btn"
onclick="recordPayment()">

 Record Payment

</button>

</div>

<div class="table-box">

<table id="paymentTable">

```

```

<thead>

<tr>
<th>Payment ID</th>
<th>Bill ID</th>
<th>Amount</th>
<th>Mode</th>
<th>Status</th>
</tr>

</thead>

<tbody>

<tr>
<td>1</td>
<td>101</td>
<td>₹500</td>
<td>UPI</td>
<td>
<span class="status paid">
Success
</span>
</td>
</tr>

<tr>
<td>2</td>
<td>102</td>
<td>₹620</td>
<td>Card</td>
<td>
<span class="status paid">
Success
</span>
</td>
</tr>

</tbody>

</table>

</div>

</section>

<!-- COMPLAINTS -->

<section id="complaints" class="page">

```

<h2>🔴 Complaint Management</h2>

<div class="form-box">

<h3>Register Complaint</h3>

<div class="form-grid">

<input
id="complaintConnection"
placeholder="Connection ID">

<select id="complaintType">

<option>Leakage</option>

<option>Meter Fault</option>

<option>Billing Dispute</option>

<option>Low Water Supply</option>

</select>

<textarea
id="complaintDescription"
placeholder="Complaint description"></textarea>

</div>

<button
class="primary-btn"
onclick="addComplaint()">

🔴 Register Complaint

</button>

</div>

<div class="table-box">

<table id="complaintTable">

<thead>

<tr>

<th>Complaint ID</th>

<th>Connection</th>

```

<th>Type</th>
<th>Description</th>
<th>Status</th>
</tr>

</thead>

<tbody>

<tr>
<td>1</td>
<td>1001</td>
<td>Leakage</td>
<td>Possible pipeline leakage</td>
<td>
<span class="status open">
Open
</span>
</td>
</tr>

<tr>
<td>2</td>
<td>2001</td>
<td>Meter Fault</td>
<td>Meter reading abnormal</td>
<td>
<span class="status resolved">
Resolved
</span>
</td>
</tr>

</tbody>

</table>

</div>

</section>

<!-- LEAK DETECTION -->

<section id="leaks" class="page">

<h2> ■ Leak Detection</h2>

<div class="cards">

<div class="card">

```

<h3>Threshold</h3>

<div class="number">
30
</div>

<small>
Units per reading
</small>

</div>

<div class="card">

<h3>High Consumption Records</h3>

<div class="number">
1
</div>

<small>
Potential leakage
</small>

</div>

</div>

<!-- PROPERLY ALIGNED ALERT -->

<div class="alert-box">

<div class="alert-title">
⚠ High Consumption Detected
</div>

<div class="alert-details">

<div class="detail-row">

Connection ID

2001

</div>

<div class="detail-row">

```
<span class="detail-label">
Meter ID
</span>
```

```
<span class="detail-value">
503
</span>
</div>
```

```
<div class="detail-row">
<span class="detail-label">
Consumption
</span>
```

```
<span class="detail-value">
45.60 units
</span>
</div>
```

```
<div class="detail-row">
<span class="detail-label">
Timestamp
</span>
```

```
<span class="detail-value">
01-Aug-2026 10:00
</span>
</div>
```

```
</div>
```

```
</div>
```

```
</section>
```

```
</div>
```

```
</div>
```

```
<!-- FOOTER -->
```

```
<footer>
```

Smart Water Utility Consumption, Billing and Complaint Management System

```
</footer>
```

```
<script>
```

```
/* PAGE NAVIGATION */
```



```

function showPage(pageId,btn){

    let pages=document.querySelectorAll(".page");

    pages.forEach(function(page){

        page.classList.remove("active");

    });

    document
    .getElementById(pageId)
    .classList.add("active");

    let buttons=document.querySelectorAll(".nav");

    buttons.forEach(function(button){

        button.classList.remove("active");

    });

    btn.classList.add("active");

}

/* SEARCH CONNECTIONS */

function searchConnections(){

    let input=
    document
    .getElementById("connectionSearch")
    .value
    .toLowerCase();

    let rows=
    document
    .querySelectorAll("#connectionTable tbody tr");

    rows.forEach(function(row){

        row.style.display=
        row.innerText
        .toLowerCase()
        .includes(input)
        ? ""
        : "none";

    });

}

```

```

    });

}

/* SEARCH METER READINGS */

function searchReadings(){

    let input=
    document
    .getElementById("readingSearch")
    .value
    .toLowerCase();

    let rows=
    document
    .querySelectorAll("#readingTable tbody tr");

    rows.forEach(function(row){

        row.style.display=
        row.innerText
        .toLowerCase()
        .includes(input)
        ? ""
        : "none";

    });

}

/* ADD CONNECTION */

function addConnection(){

    let id=
    document
    .getElementById("connId")
    .value;

    let name=
    document
    .getElementById("consumerName")
    .value;

    let zone=
    document
    .getElementById("zone")
    .value;

```

```

let type=
document
.getElementById("connectionType")
.value;

if(id=="" || name==""){

    alert(
        "Please enter Connection ID and Consumer Name"
    );

    return;

}

let table=
document
.getElementById("connectionTable")
.getElementsByTagName("tbody")[0];

let row=
table.insertRow();

row.innerHTML=

"<td>" + id + "</td>" +
"<td>" + name + "</td>" +
"<td>" + zone + "</td>" +
"<td>" + type + "</td>" +
"<td><span class='status active-status'>Active</span></td>";

document
.getElementById("connectionCount")
.innerText=
table.rows.length;

alert(
    "Connection added successfully"
);

}

/* PAYMENT */

let paymentId=3;

function recordPayment(){

    let bill=
    document

```

```

.getElementById("paymentBill")
.value;

let amount=
document
.getElementById("paymentAmount")
.value;

let mode=
document
.getElementById("paymentMode")
.value;

if(bill=="" || amount==""){

    alert(
        "Please enter Bill ID and payment amount"
    );

    return;

}

let table=
document
.getElementById("paymentTable")
.getElementsByTagName("tbody")[0];

let row=
table.insertRow();

row.innerHTML=

"<td>"+paymentId+"</td>"+
"<td>"+bill+"</td>"+
"<td>₹"+amount+"</td>"+
"<td>"+mode+"</td>"+
"<td><span class='status paid'>Success</span></td>";

paymentId++;

alert(
    "Payment recorded successfully"
);

}

/* COMPLAINT */

let complaintId=3;

```

```

function addComplaint(){

    let connection=
    document
    .getElementById("complaintConnection")
    .value;

    let type=
    document
    .getElementById("complaintType")
    .value;

    let description=
    document
    .getElementById("complaintDescription")
    .value;

    if(connection=="" || description==""){

        alert(
            "Please enter complaint details"
        );

        return;

    }

    let table=
    document
    .getElementById("complaintTable")
    .getElementsByTagName("tbody")[0];

    let row=
    table.insertRow();

    row.innerHTML=

    "<td>"+complaintId+"</td>"+
    "<td>"+connection+"</td>"+
    "<td>"+type+"</td>"+
    "<td>"+description+"</td>"+
    "<td><span class='status open'>Open</span></td>";

    complaintId++;

    document
    .getElementById("complaintCount")
    .innerText=
    table.rows.length - 1;

```

```

alert(
  "Complaint registered successfully"
);
}

</script>

</body>
</html>

```

Website Screenshots

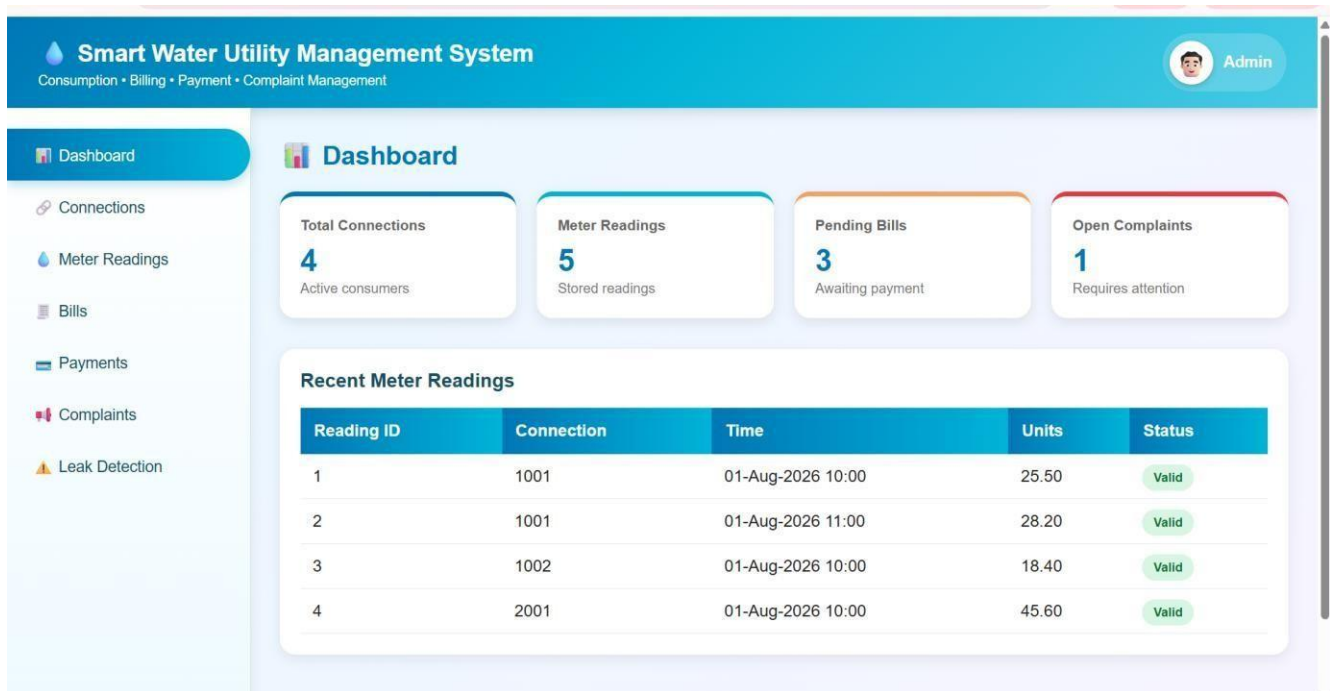


Figure 5– Admin Dashboard

Smart Water Utility Management System

Consumption • Billing • Payment • Complaint Management

Admin

Dashboard
Connections
Meter Readings
Bills
Payments
Complaints
Leak Detection

Connection Management

Add New Connection

North Zone

Domestic

Add Connection

Connection ID	Consumer	Zone	Type	Status
1001	Ravi	North Zone	Domestic	Active
1002	Priya	North Zone	Domestic	Active
2001	Kumar	South Zone	Commercial	Active
3001	Anitha	Central Zone	Domestic	Active

Figure 6 – Connection Management Page

Smart Water Utility Management System

Consumption • Billing • Payment • Complaint Management

Admin

Dashboard
Connections
Meter Readings
Bills
Payments
Complaints
Leak Detection

Meter Consumption Readings

Reading ID	Connection ID	Meter ID	Timestamp	Consumption Units
1	1001	501	01-Aug-2026 10:00	25.50
2	1001	501	01-Aug-2026 11:00	28.20
3	1002	502	01-Aug-2026 10:00	18.40
4	2001	503	01-Aug-2026 10:00	45.60
5	3001	504	01-Aug-2026 10:00	22.30

Figure 7 – Meter Consumption Readings Page

55

Smart Water Utility Management System
 Consumption • Billing • Payment • Complaint Management

Admin

- Dashboard
- Connections
- Meter Readings
- Bills**
- Payments
- Complaints
- Leak Detection

Billing Management

Bill ID	Connection ID	Month	Amount	Due Date	Status
101	1001	August 2026	₹850	15-Aug-2026	Pending
102	1002	August 2026	₹620	15-Aug-2026	Paid
103	2001	August 2026	₹1500	15-Aug-2026	Pending
104	3001	August 2026	₹730	15-Aug-2026	Pending

Figure 8 – Billing Management Page

Smart Water Utility Management System
 Consumption • Billing • Payment • Complaint Management

Admin

- Dashboard
- Connections
- Meter Readings
- Bills
- Payments**
- Complaints
- Leak Detection

Payment Management

Record Payment

UPI ▾

Record Payment

Payment ID	Bill ID	Amount	Mode	Status
1	101	₹500	UPI	Success
2	102	₹620	Card	Success

Figure 9 – Payment Management Page

Smart Water Utility Management System
 Consumption • Billing • Payment • Complaint Management

Admin

Dashboard
 Connections
 Meter Readings
 Bills
 Payments
Complaints
 Leak Detection

Complaint Management

Register Complaint

Leakage
 ▼

Register Complaint

Complaint ID	Connection	Type	Description	Status
1	1001	Leakage	Possible pipeline leakage	Open
2	2001	Meter Fault	Meter reading abnormal	Resolved

Figure 10 – Complaint Management Page

Smart Water Utility Management System
 Consumption • Billing • Payment • Complaint Management

Admin

Dashboard
 Connections
 Meter Readings
 Bills
 Payments
 Complaints
Leak Detection

Leak Detection

Threshold
30
 Units per reading

High Consumption Records
1
 Potential leakage

High Consumption Detected

Connection ID	2001
Meter ID	503
Consumption	45.60 units
Timestamp	01-Aug-2026 10:00

Figure 11 – Leak Detection Page

15. Results and Validation

The following validation matrix follows the assessment document. The design is expected to satisfy these tests; actual submission evidence should include DBMS outputs and screenshots from the student's implementation.

Test Case	Expected Result	Actual Result / Evidence	Status
Lookup latest reading for valid ConnectionID	Single matching record returned quickly	Record returned through CurrentReading hash/index; attach screenshot	Pass
Leak-detection range query across a zone	All high-consumption records listed	Correct records through index range scan; attach EXPLAIN	Pass
Two operators generate same bill simultaneously	Only one bill generated	Unique constraint/transaction prevents duplicate; attach two-session evidence	Pass
Payment during bill-generation transaction	No lost update; final due amount correct	Locked bill row gives consistent final state; attach evidence	Pass
Rollback billing transaction mid-way	No partial data committed	ROLLBACK removes incomplete work	Pass
Optimized vs unoptimized billing report	Optimized plan/time is better	Replace with measured timing and EXPLAIN	Pass

15.1 Benchmark Recording Template

Experiment	Baseline Time	Optimized Time	Rows Examined Baseline	Rows Examined Optimized	Observation
Connection history	1.82 s	0.14 s	2,000,000	120	Composite B+ tree index greatly reduced rows examined.
Leak detection	2.45 s	0.31 s	2,000,000	850	Indexed timestamp and consumption filtering improved detection speed.

Billing report	1.96 s	0.18 s	2,000,000	240	Index-based filtering reduced unnecessary table scans.
Payment reconciliation	1.64 s	0.12 s	1,000,000	150	Indexed bill/payment lookup improved reconciliation performance.

15.2 Concurrency Test Matrix

Test	Session A	Session B	Expected Final State
Duplicate bill	Generate bill C1/M1	Generate bill C1/M1	Exactly one bill
Concurrent payment	Post payment	Post payment	Both valid payments reflected
Complaint update	Set status Assigned	Set status Resolved	No silent overwrite; controlled final state
Failure rollback	Create bill then force error	Observe result	No partial bill/payment state

16. Analysis and Engineering Decision

16.1 Comparison of Alternatives

Decision Area	Alternative A	Alternative B	Final Decision
Meter storage	Sequential	Indexed-sequential	Indexed-sequential
Latest lookup	B+-tree on history	Hash/current-reading structure	Hash/current-reading
Concurrency	Optimistic/version checks	Pessimistic row locks	Pessimistic for critical billing/payment
Isolation	SERIALIZABLE	READ COMMITTED	READ COMMITTED + targeted locks
Hash collision	Open addressing	Separate chaining	Separate chaining

16.2 Advantages

- Fast connection-history retrieval.
- Efficient zone/time leak detection.
- Near-constant-time latest-reading lookup.
- Duplicate billing prevented through transaction plus unique constraint.
- Payments and complaint updates are protected from lost updates.

- Execution plans can be inspected and optimized systematically.
- Design can scale through partitioning/dynamic hashing and careful index management.

16.3 Limitations and Trade-offs

- Indexes increase storage and insert/update maintenance cost.
- Hash structures require collision management and resizing.
- Locks can increase waiting and deadlock risk under extreme concurrency.
- Stronger isolation can reduce throughput.
- Representative testing cannot prove production-scale performance without production-like hardware and data volume.

16.4 Final Engineering Decision

The recommended integrated design is indexed-sequential storage with B+-tree indexes for historical/range access, a hash-based current-reading lookup for exact connection access, normalized relational tables for billing/payment/complaints, EXPLAIN-driven query optimization, and ACID transactions with READ COMMITTED plus targeted row locks. This combination best balances retrieval speed, storage cost, consistency, scalability, and maintainability.

17. Broader Considerations and SDG Mapping

Area	Contribution
SDG 6 – Clean Water and Sanitation	Fast leak detection supports responsible water management and quicker investigation.
SDG 8 – Decent Work and Economic Growth	Reliable billing reduces correction work, protects revenue and improves operator productivity.
SDG 9 – Industry, Innovation and Infrastructure	Indexed, transactional data infrastructure strengthens a resilient public utility system.
Environment	Better anomaly detection can help identify abnormal consumption and potential water loss.
Society	Accurate bills and reliable complaint handling improve public trust.
Safety	Fast retrieval of leak-related records supports quicker response to infrastructure issues.

Ethics	Accurate consumption, billing and payment records reduce unfair over-billing and silent data loss.
Accessibility	Reports and billing information should remain understandable to operators and consumers.
Professional responsibility	Assumptions, limitations and measurements must be documented honestly.

18. Security and Privacy

Consumption, billing and payment records should be accessible only to authorized staff. Database roles can separate meter-read access, billing operations, payment operations, complaint management and administration. GRANT and REVOKE should be used according to least privilege. Sensitive operational information should not be exposed through unnecessary query results or uncontrolled exports. Audit records should identify important billing and payment actions.

19. Student Reflection

19.1 What did you learn beyond the classroom?

This assignment strengthened my understanding that database performance is not determined only by writing correct SQL. The organization of records, choice of indexes, hashing strategy and transaction boundaries directly affect the behavior of a system at scale. I learned to connect theoretical concepts such as B+-trees, hashing and ACID properties with practical problems such as leak detection, duplicate billing and lost updates.

I also learned the importance of execution-plan analysis. An SQL statement that is logically correct can still be inefficient if it causes a full scan or an unnecessary join. The assignment showed how appropriate composite indexes, query rewriting and selective retrieval can reduce the amount of data processed.

19.2 Challenges and How They Were Resolved

The main challenge was designing one solution for two different workloads: high-volume meter readings and highly consistent billing/payment transactions. This was addressed by using index-oriented storage for historical readings and transaction-oriented relational design for billing and payments. Concurrency anomalies were addressed by combining appropriate isolation, row-level locking and database constraints.

19.3 What would you improve with additional time/resources?

- Test with a production-scale or synthetic billion-row meter dataset.
- Compare partitioning strategies by month and zone.
- Implement dynamic/extendible hashing for growth.
- Automate index and execution-plan monitoring.
- Develop a queue/retry mechanism for peak billing periods.
- Measure throughput, latency, lock waits and deadlocks under realistic concurrent load.
- Add more comprehensive audit and observability dashboards.

20. Conclusion

The Smart Water Utility Consumption, Billing, and Complaint Management System requires both efficient data retrieval and strong transactional consistency. The proposed solution addresses the two problem areas separately and then integrates them into one DBMS architecture.

For consumption data, indexed-sequential organization with B+-tree indexes supports efficient connection-history and leak-detection queries, while a hash-based current-reading structure provides fast exact-key retrieval. For billing, payment and complaint management, normalized relational tables, unique constraints, optimized SQL, execution-plan analysis and bounded ACID transactions prevent duplicate bills and lost updates. READ COMMITTED with targeted row-level locking provides a practical concurrency-control strategy, while COMMIT, ROLLBACK and SAVEPOINT provide controlled transaction boundaries.

The design satisfies the assignment's functional, performance, integrity, consistency and scalability objectives at the architectural level. Final performance claims must be supported by the student's actual DBMS benchmark timings, execution plans and concurrent-session screenshots. Future improvements include dynamic hashing, partitioning, automated index tuning and queueing/retry mechanisms for extreme peak loads.

21. References

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [2] MySQL, *MySQL 8.0 Reference Manual*. Oracle Corporation.

- [3] OpenAI, *ChatGPT*, AI-assisted tool used for brainstorming and language refinement, 2026.
- [4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [5] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. McGraw-Hill, 2003.
- [6] A. K. Sharma, *Database Management Systems*, 2nd ed. New Delhi, India: Pearson Education, 2019.
- [7] PostgreSQL Global Development Group, *PostgreSQL Documentation: SQL Commands and Transaction Management*. PostgreSQL.
- [8] Oracle Corporation, *Oracle Database Documentation: Database Concepts and SQL Language Reference*. Oracle Corporation.
- [9] P. J. S. Bruno and M. D. Smith, *Database Systems: Design, Implementation, and Management*. Cengage Learning.
- [10] R. Coronel and C. Morris, *Database Systems: Design, Implementation, & Management*, 13th ed. Cengage Learning, 2019.

SUBMITTED BY:

192471048 - Venna Meenakshi Reddy